

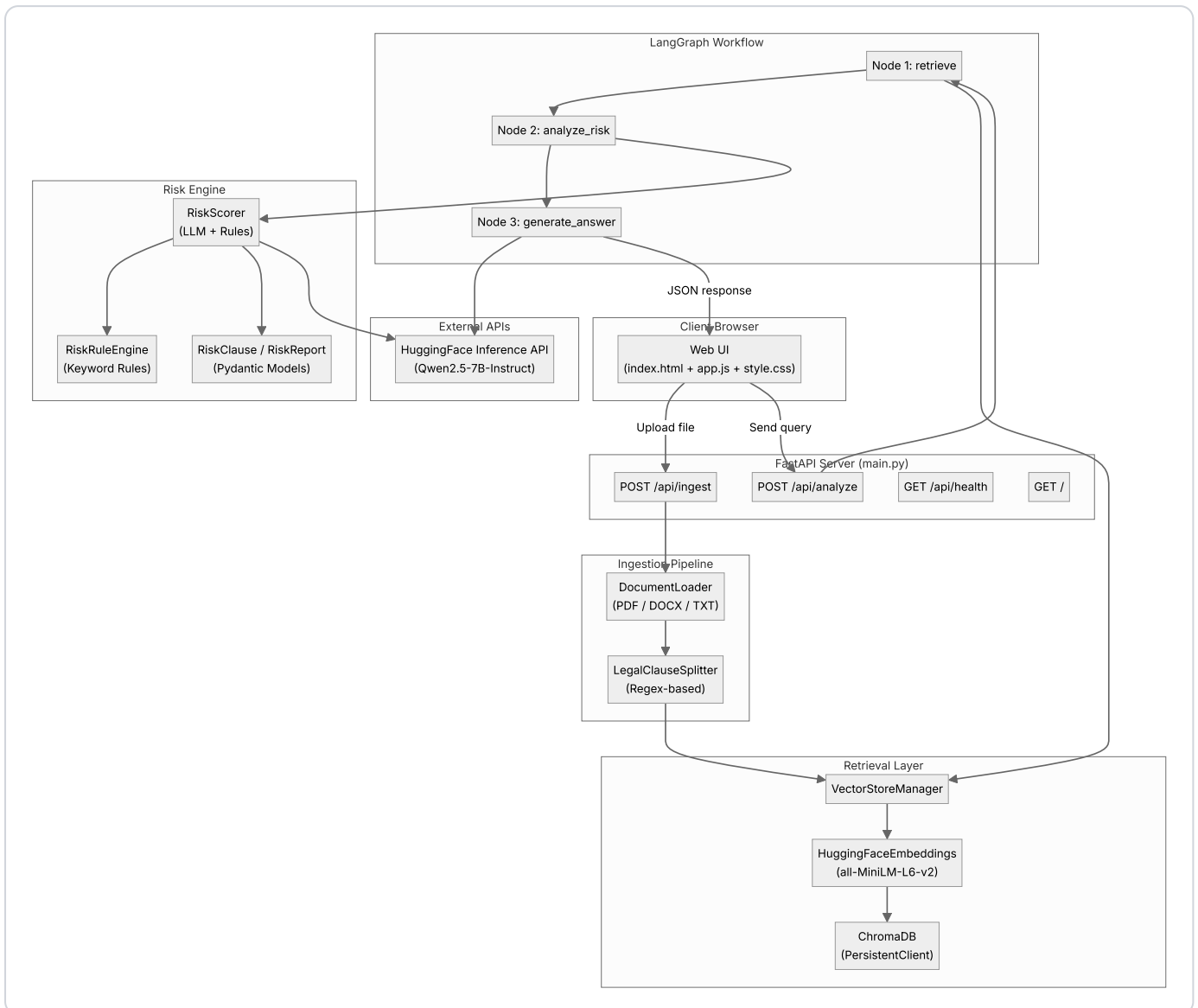
AI Legal Policy & Document Analyzer — Complete Project Report

1. Project Overview

The **AI Legal Policy and Document Analyzer** is a full-stack web application that uses Retrieval-Augmented Generation (RAG) and a hybrid risk-scoring engine to analyze legal contracts. A user uploads a legal document (PDF, DOCX, or TXT), the system splits it into semantic clauses, embeds them into a vector database, and then answers natural-language questions about the contract while simultaneously producing a per-clause risk assessment.

Attribute	Value
Language	Python 3.11
Framework	FastAPI (async REST API)
AI Stack	LangChain + LangGraph + HuggingFace LLMs
Vector DB	ChromaDB (local persistent)
Embeddings	<code>all-MiniLM-L6-v2</code> (Sentence-Transformers)
LLM	<code>Qwen/Qwen2.5-7B-Instruct</code> (via HuggingFace Inference API)
Frontend	Vanilla HTML / CSS / JavaScript
Deployment	Docker → Hugging Face Spaces
CI/CD	GitHub Actions (<code>ci.yml</code> + <code>deploy.yml</code>)

2. High-Level Architecture



3. Repository Structure

```
AI-Legal-Policy-and-Document-Analyzer/
├── .env                                # API keys (git-ignored)
├── .gitignore                          # Exclusion rules
├── .github/
│   └── workflows/
│       ├── ci.yml                    # CI pipeline (pytest on push/PR)
│       └── deploy.yml                # CD pipeline (sync to HF Spaces)
├── Dockerfile                         # Container config for HF Spaces
├── README.md                          # Project readme
├── RISK_ANALYSIS.md                   # Risk analysis documentation
├── architecture_and_design.md         # Design notes
├── pyproject.toml                     # pytest-asyncio configuration
├── requirements.txt                   # All dependencies (runtime + dev)
├──
├── main.py                           # FastAPI app (primary entry point)
├── web_server.py                      # Alternative dev server (port 8001)
├──
├── src/
│   ├── __init__.py
│   ├── ingestion/
│   │   ├── __init__.py
│   │   ├── ingestion_loader.py       # DocumentLoader (PDF/DOCX/TXT)
│   │   └── legal_splitter.py         # LegalClauseSplitter (regex)
│   ├── retrieval/
│   │   ├── __init__.py
│   │   └── vector_storage.py         # VectorStoreManager (ChromaDB)
│   ├── risk_engine/
│   │   ├── __init__.py
│   │   ├── risk_models.py            # RiskClause, RiskReport (Pydantic)
│   │   ├── risk_rules.py             # RiskRuleEngine (keyword rules)
│   │   └── risk_scorer.py            # RiskScorer (LLM + rules hybrid)
│   ├── utils/
│   │   ├── __init__.py
│   │   └── project_config.py         # Config (env vars, model names)
│   └── workflows/
│       ├── __init__.py
│       ├── workflow_graph.py         # LangGraph StateGraph builder
│       └── workflow_nodes.py         # Node functions + GraphState
├──
├── web/
│   ├── index.html                    # Single-page frontend
│   └── static/
│       ├── css/style.css             # UI styling (dark theme)
│       └── js/app.js                 # Frontend logic (fetch API)
├──
├── samples/                           # Sample legal documents
│   ├── saas_contract.txt
│   └── sample_nda.txt
```

```

|   ├── sample_privacy_policy.txt
|   └── sample_vendor_agreement.txt
|
└── tests/
    ├── __init__.py
    ├── conftest.py           # Shared pytest fixtures
    ├── test_api_endpoints.py # API endpoint tests (7 tests)
    ├── test_risk_engine.py   # Risk engine tests (13 tests)
    ├── test_splitter.py      # Splitter tests (9 tests)
    ├── test_workflow_graph.py # Workflow tests (5 tests)
    └── test_scenarios.py     # Manual integration script

```

4. Component-by-Component Deep Dive

4.1 Configuration Layer

[src/utils/project_config.py](#)

The `Config` class is the single source of truth for all environment-specific settings. It loads variables from the `.env` file using `python-dotenv` and exposes them as class attributes.

Setting	Value	Purpose
<code>HUGGINGFACEHUB_API_TOKEN</code>	From <code>.env</code>	Authenticates with HuggingFace Inference API
<code>CHROMA_PERSIST_DIRECTORY</code>	<code><project_root>/chroma_db/</code>	Local directory for ChromaDB persistence
<code>EMBEDDING_MODEL</code>	<code>all-MiniLM-L6-v2</code>	Sentence-Transformer model for clause embeddings
<code>LLM_SCAN_MODEL</code>	<code>Qwen/Qwen2.5-7B-Instruct</code>	LLM used for risk analysis (batch clause scanning)
<code>LLM_REASONING_MODEL</code>	<code>Qwen/Qwen2.5-7B-Instruct</code>	LLM used for final answer generation

`validate_api_key()` — A classmethod that raises `ValueError` if the token is missing, empty, or a placeholder. Called at the start of every `/api/ingest` request.

`.env` (git-ignored)

```

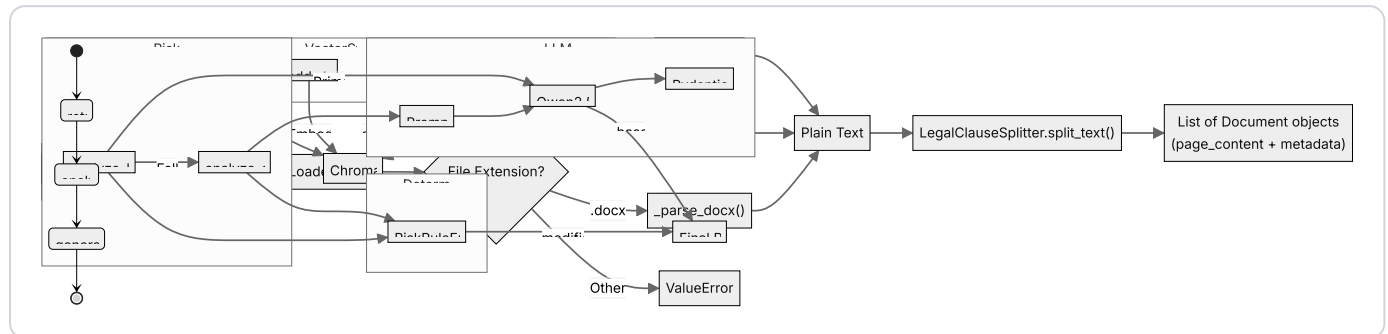
GOOGLE_API_KEY=<key>
HUGGINGFACEHUB_API_TOKEN=<token>

```

Sensitive credentials are isolated here and excluded from version control via `.gitignore`.

4.2 Ingestion Pipeline

The ingestion pipeline converts raw uploaded files into semantically chunked `Document` objects ready for vector storage.



[src/ingestion/ingestion_loader.py](#) — `DocumentLoader`

A static utility class that normalises any supported file format into plain text.

Method	Input	Output	Logic
<code>load(file_obj, filename)</code>	<code>bytes</code> or <code>BinaryIO</code> + filename string	<code>str</code> (full text)	Routes to format-specific parser based on file extension
<code>_parse_txt(file_obj)</code>	<code>BinaryIO</code>	<code>str</code>	Decodes as UTF-8, falls back to Latin-1
<code>_parse_pdf(file_obj)</code>	<code>BinaryIO</code>	<code>str</code>	Uses <code>pypdf.PdfReader</code> to extract text from every page
<code>_parse_docx(file_obj)</code>	<code>BinaryIO</code>	<code>str</code>	Uses <code>python-docx</code> to join all paragraph texts

Error handling: Raises `ValueError` for unsupported formats. PDF/DOCX parsing failures are caught and re-raised as `ValueError` with descriptive messages.

[src/ingestion/legal_splitter.py](#) — `LegalClauseSplitter`

A custom `TextSplitter` (extends LangChain's base class) that uses **compiled regex patterns** to split legal text on clause boundaries.

Recognised patterns (compiled with `re.IGNORECASE | re.MULTILINE`):

Pattern	Example Match
<code>^\s*ARTICLE\s+[IVX0-9]+</code>	ARTICLE I , ARTICLE IV , ARTICLE 3
<code>^\s*SECTION\s+[0-9]+(\.[0-9]+)*</code>	SECTION 1 , SECTION 2.3
<code>^\s*[0-9]+\.[0-9]+(\.[0-9]+)*</code>	1.1 , 2.3.1 , 10.4.2
<code>^\s*[0-9]+\.\s+[A-Z]</code>	3. Definitions , 1. Term

`split_text(text)` — Iterates through lines; when a line matches a pattern it starts a new chunk. Returns `List[str]`.

`create_documents(texts, metadatas)` — Wraps `split_text()` and produces `List[Document]` objects. Each document gets:

- `page_content`: the clause text
- `metadata.clause_id`: extracted from the regex match (e.g., `"1.1"`, `"ARTICLE II"`) or `"Intro"` if no pattern matched
- Any additional metadata from the caller (e.g., `{"source": "contract.pdf"}`)

4.3 Retrieval Layer (Vector Storage)

`src/retrieval/vector_storage.py` — `VectorStoreManager`

Manages the ChromaDB vector store for semantic similarity search over legal clauses.

Method	Input	Output	Description
<code>__init__()</code>	—	—	Creates embeddings model, ChromaDB persistent client, and Chroma collection. Auto-resets DB if initialisation fails.
<code>add_documents(docs, clear_existing)</code>	<code>List[Document]</code> , <code>bool</code>	<code>List[str]</code> (IDs)	Embeds and stores documents. Optionally clears existing data first.
<code>search(query, k)</code>	Query <code>str</code> , top-k <code>int</code>	<code>List[(Document, float)]</code>	Returns top-k most similar clauses with cosine similarity scores.
<code>clear_all()</code>	—	—	Drops and recreates the <code>legal_clauses</code> collection.
<code>get_retriever(k)</code>	top-k <code>int</code>	<code>VectorStoreRetriever</code>	Returns a LangChain-compatible retriever interface.

Key design: The collection name is hardcoded as `legal_clauses`. When a new document is ingested, `clear_existing=True` wipes the previous document's embeddings so the vector store always represents the most recently ingested document.

4.4 Risk Engine

The risk engine is the analytical core. It combines LLM-based clause analysis with a deterministic keyword-based rule engine for a **hybrid scoring approach**.

[src/risk_engine/risk_models.py](#) — **Pydantic Data Models**

RiskClause — The structured output for a single analysed clause:

Field	Type	Constraints	Description
<code>clause_id</code>	<code>str</code>	Required	Clause number or identifier (e.g., <code>"1.1"</code> , <code>"Section 5"</code>)
<code>clause_type</code>	<code>str</code>	Required	Category (e.g., <code>"Indemnity"</code> , <code>"Liability Cap"</code> , <code>"Termination"</code>)
<code>risk_level</code>	<code>str</code>	Required	<code>"High"</code> , <code>"Medium"</code> , or <code>"Low"</code>
<code>risk_score</code>	<code>int</code>	<code>ge=1,</code> <code>le=10</code>	Numeric risk score (1 = lowest, 10 = highest)
<code>reason</code>	<code>str</code>	Required	Plain-English explanation of the risk
<code>recommendation</code>	<code>str</code>	Required	Actionable advice to mitigate the risk

RiskReport — Aggregate report for an entire document:

Field	Type	Description
<code>document_id</code>	<code>str</code>	Document identifier
<code>overall_risk_score</code>	<code>float</code>	Weighted average of clause scores
<code>high_risk_clauses</code>	<code>List[RiskClause]</code>	All clauses scoring ≥ 8
<code>medium_risk_clauses</code>	<code>List[RiskClause]</code>	All clauses scoring 5–7
<code>low_risk_clauses</code>	<code>List[RiskClause]</code>	All clauses scoring 1–4

[src/risk_engine/risk_rules.py](#) — **RiskRuleEngine**

A deterministic, keyword-based scoring system that acts as a **safety net** alongside LLM analysis. Ensures critical risk patterns are never missed, even if the LLM hallucinates.

Required Keywords	Score Modifier	Label	Risk Rationale
<code>indemnify</code> + <code>unlimited</code>	+7	Unlimited indemnity	Unrestricted financial exposure
<code>liability</code> + <code>cap</code>	-2	Liability cap present	Risk is mitigated by a cap
<code>termination</code> + <code>convenience</code>	+3	Termination for convenience	Contract can be ended without cause
<code>auto-renew</code>	+2	Auto-renewal clause	May lock into unwanted extensions
<code>confidential</code> + <code>survival</code>	+1	Confidentiality survival	Obligations persist after termination
<code>no warranty</code> + <code>as is</code>	+2	Warranty disclaimer	No quality guarantees
<code>governing law</code>	-1	Governing law defined	Legal clarity reduces ambiguity

`evaluate(text)` — Scans text (case-insensitive) against all rules. Returns `(total_modifier: int, triggered_labels: List[str])`.

[src/risk_engine/risk_scorer.py](#) — `RiskScorer`

The main scorer that orchestrates LLM-based analysis and deterministic rule evaluation.

Two analysis modes:

Mode	Method	When Used	API Calls
Batch (preferred)	<code>analyze_batch(clauses)</code>	Always attempted first	1 LLM call for all clauses
Single (fallback)	<code>analyze_clause(id, text)</code>	Only if batch JSON parsing fails	1 LLM call per clause

Batch analysis flow:

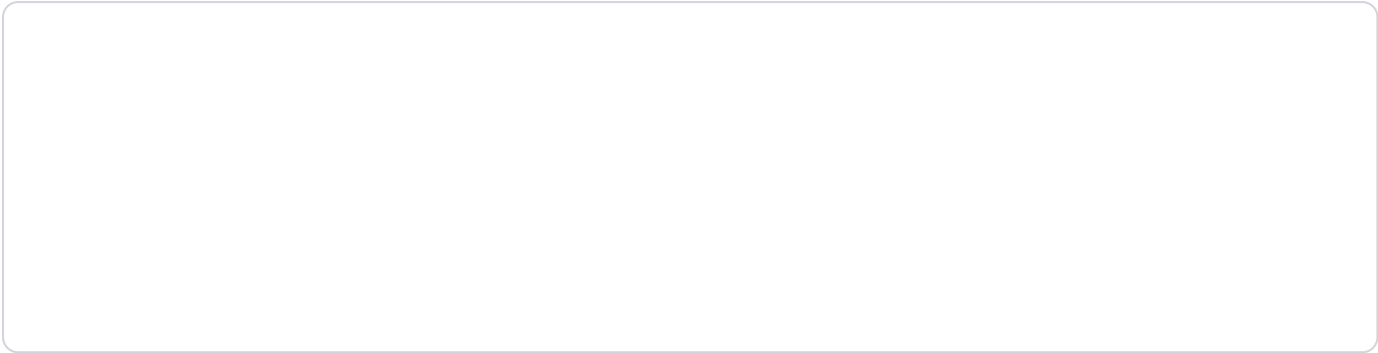
1. Format all clauses into a single prompt requesting a JSON array.
2. Send to Qwen2.5-7B-Instruct via HuggingFace Inference API.
3. Extract the JSON array from the response (strips markdown fences if present).
4. Parse each item into a `RiskClause` object.
5. Apply `RiskRuleEngine.evaluate()` to each clause's original text — adjusts the score and appends triggered rule labels.
6. Clamp final score to `[1, 10]` and recalculate risk level: `≥8 → High`, `≥5 → Medium`, `<5 → Low`.

Fallback: If batch parsing fails (malformed JSON, API error), falls back to `asyncio.gather()` over individual `analyze_clause()` calls.

4.5 LangGraph Workflow (State Machine)

The workflow orchestrates the three-stage analysis pipeline as a **directed acyclic graph** using LangGraph's `StateGraph`.

[src/workflows/workflow_graph.py](#)



`create_workflow()` builds and compiles the graph:

1. Creates a `StateGraph(GraphState)`.
2. Adds three nodes: `retrieve`, `analyze_risk`, `generate_answer`.
3. Sets `retrieve` as the entry point.
4. Wires linear edges: `retrieve → analyze_risk → generate_answer → END`.
5. Returns the compiled graph (supports both `invoke()` and `ainvoke()`).

[src/workflows/workflow_nodes.py](#) — `GraphState` & `LegalNodes`

`GraphState` — A `TypedDict` defining the data that flows through the graph:

Key	Type	Purpose
<code>query</code>	<code>str</code>	User's natural-language question
<code>documents</code>	<code>list</code>	Retrieved <code>Document</code> objects
<code>risk_analysis</code>	<code>List[Any]</code>	List of <code>RiskClause</code> results
<code>final_answer</code>	<code>str</code>	Generated plain-English answer
<code>overall_report</code>	<code>Dict[str, Any]</code>	Aggregate risk statistics

`LegalNodes` — Contains the three async node functions:

Node	Function	Input State Keys	Output State Keys	What It Does
retrieve	LegalNodes.retrieve()	query	documents	Runs <code>VectorStore.search(query, k=5)</code> to get relevant clauses
analyze_risk	LegalNodes.analyze_risk()	documents	risk_analysis	Passes clauses to <code>RiskScore.analyze()</code> which returns list of risks
generate_answer	LegalNodes.generate_answer()	risk_analysis, query	final_answer, overall_report	Builds a prompt for clause analysis. Instruct, retrieve relevant clauses, return answer

Answer generation prompt structure:

You are a helpful and expert legal document assistant.

User question: '{query}'

--- Document Risk Statistics ---

Overall Risk Score: X/10

Critical (High) Risks: N

Medium Risks: N

Low Risks: N

--- Clause Analysis ---

- Clause 1.1 (High, Score: 8/10): Reason

Recommendation: Advice

...

--- Instructions ---

1. Identify document type

2. Use plain language

3-6. Formatting and response rules

4.6 FastAPI REST API

`main.py` — Primary Application Entry Point

Endpoint	Method	Input	Output	Description
<code>/</code>	GET	—	<code>HTMLResponse</code>	Serves the frontend (<code>web/index.html</code>)
<code>/api/ingest</code>	POST	<code>UploadFile</code> (multipart form)	<code>{"status", "num_clauses", "filename", "message"}</code>	Validates API key, parses file, splits into clauses, stores in ChromaDB
<code>/api/analyze</code>	POST	<code>{"query": str}</code> (JSON body)	<code>{"status", "answer", "overall_report", "num_clauses_analyzed"}</code>	Runs the full LangGraph workflow and returns the answer
<code>/api/health</code>	GET	—	<code>{"status": "ok", "version": "1.0.0"}</code>	Health check for monitoring
<code>/static/*</code>	GET	—	Static files	Serves CSS and JS assets

Error handling:

- Unsupported file formats → `400` with descriptive error.
- API quota exhaustion (HTTP 429) → Graceful `200` with a user-friendly quota-exceeded message.
- All other exceptions → `500` with traceback logging.

`web_server.py` — Development Server

An identical copy of `main.py` with an added `if __name__ == "__main__"` block that launches Uvicorn on port 8001. Used for local development.

4.7 Frontend (Web UI)

`web/index.html`

A single-page application with three sections:

Section	ID	Purpose
Upload Contract	<code>uploadArea</code>	Drag-and-drop or click-to-browse file upload (TXT, PDF, DOCX)
Analyze Contract	<code>queryInput</code>	Text area for natural-language queries + 6 pre-built quick-query buttons
Analysis Results	<code>resultsSection</code>	Displays risk summary badges + formatted answer text

Quick-query presets:

-  High-Risk Clauses
-  Termination Period
-  Indemnification

-  Liability Limits
-  Auto-Renewal
-  Data Protection

[web/static/js/app.js](#)

Vanilla JavaScript handling all UI interactions:

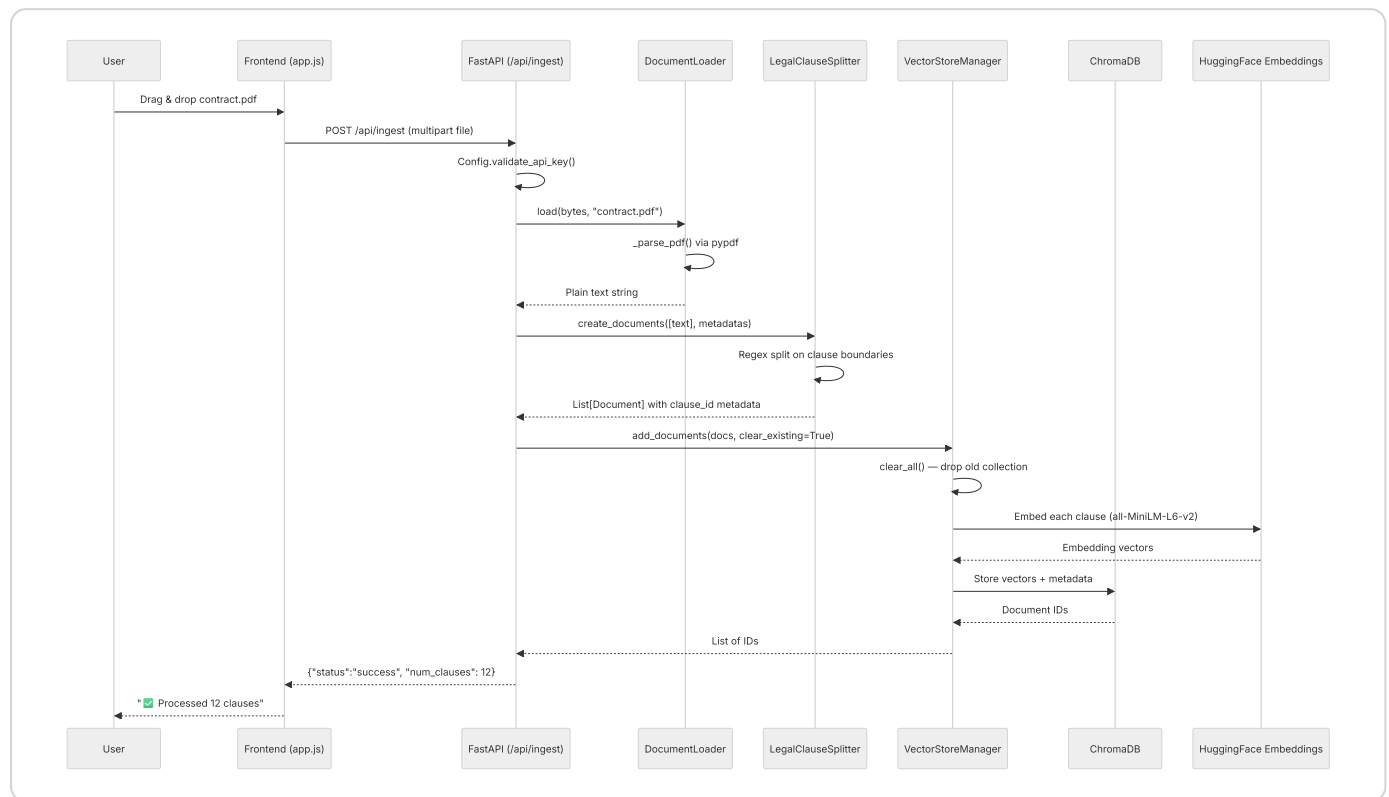
Function	Trigger	What It Does
<code>handleFileSelection(file)</code>	File input change / drag-drop	Stores file reference, updates UI, enables ingest button
<code>handleIngestion()</code>	"Ingest Document" click	<code>POST /api/ingest</code> with <code>FormData</code> , shows success/error
<code>handleAnalysis()</code>	"Analyze" click or Ctrl+Enter	<code>POST /api/analyze</code> with JSON query, renders results
<code>renderResults(data)</code>	After successful analysis	Builds risk-summary badges (score, high/medium/low counts) and formats the answer
<code>formatAnswer(text)</code>	During render	Converts <code>**bold**</code> → <code></code> , <code>*italic*</code> → <code></code> , newlines → <code>
</code>

[web/static/css/style.css](#)

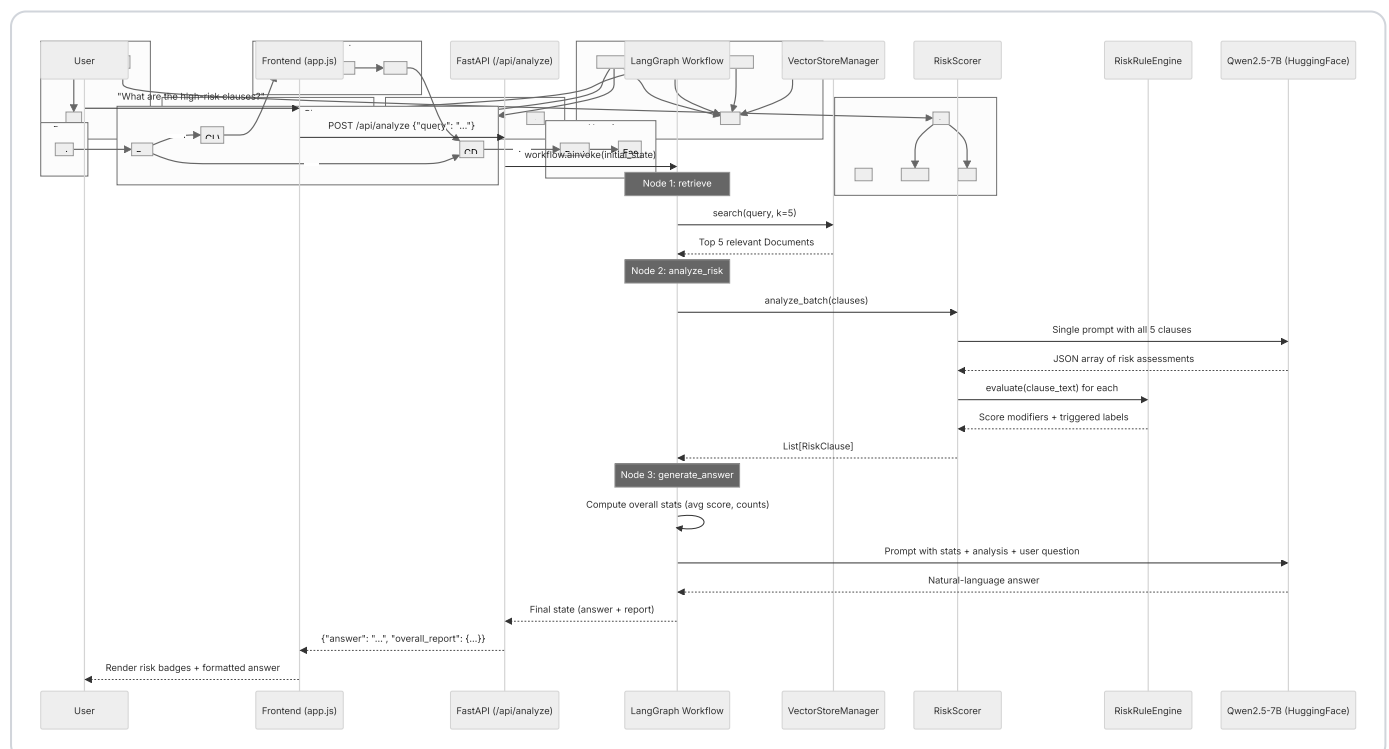
7,730 bytes of custom CSS implementing a dark-themed, modern UI with Inter font, glassmorphism cards, and responsive layout.

5. Data Flow: End-to-End

5.1 Document Ingestion Flow



5.2 Analysis Query Flow



6. Input / Output Specifications

6.1 Inputs

Input	Source	Format	Constraints
Legal document	File upload via UI	<code>.txt</code> , <code>.pdf</code> , <code>.docx</code>	Any size; single file at a time
Analysis query	Text input via UI	Free-text <code>str</code>	Non-empty string
API keys	<code>.env</code> file or GitHub Secrets	Environment variables	<code>HUGGINGFACEHUB_API_TOKEN</code> (required), <code>GOOGLE_API_KEY</code> (optional)

6.2 Outputs

Ingestion response:

```
{
  "status": "success",
  "num_clauses": 12,
  "filename": "contract.pdf",
  "message": "Successfully processed 12 clauses from 'contract.pdf'"
}
```

Analysis response:

```
{
  "status": "success",
  "answer": "This SaaS agreement contains **3 high-risk clauses**...",
  "overall_report": {
    "overall_risk_score": 6.8,
    "high_risk_count": 3,
    "medium_risk_count": 1,
    "low_risk_count": 1
  },
  "num_clauses_analyzed": 5
}
```

Risk clause structure (internal, per clause):

```
{
  "clause_id": "3.2",
  "clause_type": "Indemnity",
  "risk_level": "High",
  "risk_score": 9,
  "reason": "Unlimited indemnification with no cap | Rules triggered: unlimited indemnity",
  "recommendation": "Negotiate a liability cap and mutual indemnification."
}
```

7. Testing — Complete Breakdown

7.1 Test Architecture

```
tests/
├─ conftest.py           ← Shared fixtures (mock_env, async_client)
├─ test_api_endpoints.py ← FastAPI endpoint integration tests
├─ test_risk_engine.py   ← RiskRuleEngine + Pydantic model tests
├─ test_splitter.py      ← LegalClauseSplitter regex tests
├─ test_workflow_graph.py ← LangGraph structure tests
└─ test_scenarios.py     ← Manual integration script (not in CI)
```

Key design principle: All tests are **fully offline** — LLM calls, embedding models, and ChromaDB are mocked. Tests run in **0.22 seconds** with zero network dependencies.

7.2 Shared Fixtures (`conftest.py`)

Fixture	Scope	Purpose
<code>mock_env</code>	<code>autouse=True</code> (every test)	Sets fake <code>HUGGINGFACEHUB_API_TOKEN</code> and <code>GOOGLE_API_KEY</code> via <code>monkeypatch.setenv()</code> so <code>Config.validate_api_key()</code> passes without real credentials
<code>async_client</code>	Per-test	Creates an <code>httpx.AsyncClient</code> with <code>ASGITransport</code> bound to the FastAPI <code>app</code> . All HTTP requests go through ASGI transport — no TCP socket, no real server.

7.3 Test Module Details

`test_api_endpoints.py` — 7 Tests

Test	What It Validates
<code>test_health_returns_ok</code>	<code>GET /api/health</code> returns 200 with <code>{"status": "ok"}</code>
<code>test_serve_home_returns_html</code>	<code>GET /</code> returns 200 with <code>text/html</code> content type
<code>test_ingest_txt_file</code>	<code>POST /api/ingest</code> with a <code>.txt</code> file returns success + clause count (mocks <code>VectorStoreManager</code> and <code>Config</code>)
<code>test_ingest_unsupported_format</code>	<code>.csv</code> upload returns 400 or 500 (not 200)
<code>test_ingest_missing_file</code>	No file body → 422 Unprocessable Entity
<code>test_analyze_returns_answer</code>	<code>POST /api/analyze</code> with mocked workflow returns the expected answer and report
<code>test_analyze_missing_query</code>	Empty JSON body → 422

Mocking strategy: `@patch("main.VectorStoreManager")`, `@patch("main.Config")`, and `@patch("main.create_workflow")` replace heavy dependencies with `MagicMock` / `AsyncMock`.

`test_splitter.py` — 9 Tests

Test	What It Validates
<code>test_split_numbered_clauses</code>	<code>"1.1 ..."</code> and <code>"1.2 ..."</code> produce 2 documents with correct <code>clause_id</code>
<code>test_split_deeply_nested_numbers</code>	<code>"1.2.3 ..."</code> is recognised as a clause boundary
<code>test_split_articles</code>	<code>"ARTICLE I: ..."</code> produces documents with <code>ARTICLE I</code> in metadata
<code>test_split_roman_numerals_articles</code>	<code>ARTICLE IV</code> , <code>ARTICLE V</code> split correctly
<code>test_intro_fallback</code>	Text without clause markers gets <code>clause_id = "Intro"</code>
<code>test_empty_text</code>	Empty string → 0 documents
<code>test_whitespace_only_text</code>	Whitespace-only → 0 documents
<code>test_metadata_preserved</code>	Custom <code>metadatas=[{"source": "test.txt"}]</code> appears in output
<code>test_multiple_texts_with_metadata</code>	Multiple texts with separate metadata lists are handled correctly

`test_risk_engine.py` — 13 Tests

`TestRiskRuleEngine` (8 tests):

Test	Rule	Expected
<code>test_unlimited_indemnity_rule</code>	<code>indemnify</code> + <code>unlimited</code>	modifier $\geq +7$, label = <code>"unlimited indemnity"</code>
<code>test_liability_cap_rule</code>	<code>liability</code> + <code>cap</code>	modifier ≤ -2 , label = <code>"liability cap present"</code>
<code>test_termination_for_convenience</code>	<code>termination</code> + <code>convenience</code>	modifier $\geq +3$
<code>test_auto_renewal_rule</code>	<code>auto-renew</code>	modifier $\geq +2$
<code>test_governing_law_rule</code>	<code>governing law</code>	modifier ≤ -1
<code>test_no_rules_triggered</code>	Benign text	modifier = 0, triggered = []
<code>test_multiple_rules_triggered</code>	<code>indemnify unlimited</code> + <code>termination convenience</code>	modifier = 10 (+7 + +3), 2 labels
<code>test_case_insensitive_matching</code>	<code>"AUTO-RENEW"</code> (uppercase)	Still triggers the rule

`TestRiskClauseModel` (5 tests):

Test	What It Validates
<code>test_valid_risk_clause</code>	A valid <code>RiskClause</code> object is created without errors
<code>test_risk_score_out_of_range</code>	<code>risk_score=15</code> → <code>ValidationError</code>
<code>test_risk_score_zero_invalid</code>	<code>risk_score=0</code> → <code>ValidationError</code> (minimum is 1)
<code>test_missing_required_field</code>	Omitting <code>clause_type</code> → <code>ValidationError</code>
<code>test_risk_report_model</code>	<code>RiskReport</code> accepts lists of <code>RiskClause</code> objects

`test_workflow_graph.py` — 5 Tests

Test	What It Validates
<code>test_graph_state_has_required_keys</code>	<code>GraphState</code> TypedDict has exactly: <code>query</code> , <code>documents</code> , <code>risk_analysis</code> , <code>final_answer</code> , <code>overall_report</code>
<code>test_graph_state_is_instantiable</code>	A valid <code>GraphState</code> dict can be constructed and accessed
<code>test_workflow_compiles</code>	<code>create_workflow()</code> returns an object with <code>invoke</code> / <code>ainvoke</code> methods
<code>test_workflow_has_expected_nodes</code>	Compiled graph contains nodes: <code>retrieve</code> , <code>analyze_risk</code> , <code>generate_answer</code>
<code>test_workflow_entry_point</code>	The <code>__start__</code> node has an edge to <code>retrieve</code>

Mocking: All 4 heavy dependencies (`VectorStoreManager` , `HuggingFaceEndpoint` , `ChatHuggingFace` , `RiskScorer`) are patched with `MagicMock` so the graph compiles without network calls.

7.4 Test Results Summary

```
===== test session starts =====
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.6.0
asyncio: mode=Mode.AUTO
plugins: asyncio-1.4.0, langsmith-0.8.9
collected 34 items

tests/test_api_endpoints.py    7 passed    ✓
tests/test_risk_engine.py     13 passed    ✓
tests/test_splitter.py         9 passed    ✓
tests/test_workflow_graph.py   5 passed    ✓

===== 34 passed, 0 failed in 0.22s =====
```

8. CI/CD Pipeline — Concepts & Configuration

8.1 What is CI/CD?

Concept	Definition	Implementation in This Project
Continuous Integration (CI)	Automatically build and test code on every push/PR	<code>ci.yml</code> — runs <code>pytest</code> on every push/PR to <code>main</code>
Continuous Deployment (CD)	Automatically deploy validated code to production	<code>deploy.yml</code> — syncs to Hugging Face Spaces on every push to <code>main</code>
GitHub Actions	GitHub's built-in automation platform; executes workflows defined in YAML	Both workflows live in <code>.github/workflows/</code>
Workflow	A configurable automated process made up of jobs and steps	Each <code>.yml</code> file defines one workflow
Job	A set of steps that execute on the same runner	<code>test</code> job in <code>ci.yml</code> , <code>deploy</code> job in <code>deploy.yml</code>
Runner	A virtual machine that executes jobs	<code>ubuntu-latest</code> (GitHub-hosted)
Trigger	Events that start a workflow	<code>push</code> to <code>main</code> , <code>pull_request</code> to <code>main</code>
GitHub Secrets	Encrypted environment variables accessible in workflows	<code>HUGGINGFACEHUB_API_TOKEN</code> , <code>GOOGLE_API_KEY</code> , <code>HF_TOKEN</code>
Caching	Storing dependencies between runs for speed	<code>actions/cache@v4</code> caches <code>~/.cache/pip</code>

8.2 CI Workflow (`ci.yml`)

```
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
```

Triggers: Runs on any push to `main` OR any pull request targeting `main`.

Steps in the `test` job:

Step	Action	Purpose
1. Checkout code	<code>actions/checkout@v4</code>	Clones the repository onto the runner
2. Set up Python	<code>actions/setup-python@v5</code>	Provisions Python 3.11 on the Ubuntu runner
3. Cache pip	<code>actions/cache@v4</code>	Caches <code>~/.cache/pip</code> keyed by <code>requirements.txt</code> hash — speeds up subsequent runs
4. Install dependencies	<code>pip install -r requirements.txt</code>	Installs all runtime + test dependencies
5. Run tests	<code>pytest tests/ -v --tb=short</code>	Executes all 34 tests with verbose output

Environment variables: `HUGGINGFACEHUB_API_TOKEN` and `GOOGLE_API_KEY` are injected from GitHub Secrets.

8.3 CD Workflow (`deploy.yml`)

```
name: Sync to Hugging Face Hub
on:
  push:
    branches: [main, master]
```

Steps:

- `actions/checkout@v4` with `fetch-depth: 0` (full history) and `lfs: true` (large file support).
- Force-pushes the entire repo to the Hugging Face Spaces remote using `HF_TOKEN`.

Deployment target: `huggingface.co/spaces/ykjaat6104/AI-Legal-Policy-and-Document-Analyzer`

8.4 Docker Deployment ([Dockerfile](#))

```
FROM python:3.11-slim
WORKDIR /code
COPY ./requirements.txt /code/requirements.txt
RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "7860"]
```

Hugging Face Spaces requires port **7860**. The Dockerfile uses a multi-stage copy strategy (requirements first) to leverage Docker layer caching.

8.5 Repository Hygiene

[.gitignore](#) — Key Exclusions

Pattern	What It Excludes	Why
<code>chroma_db/</code>	Local ChromaDB database files	Binary data, regenerated on ingestion
<code>.env</code>	API keys and tokens	Security — never commit secrets
<code>__pycache__/</code> , <code>*.pyc</code>	Python bytecode	Generated artifacts
<code>.pytest_cache/</code>	Pytest cache	Temporary test data
<code>*.bin</code> , <code>*.safetensors</code> , <code>*.onnx</code> , <code>*.h5</code>	Model weight files	Massive binaries (100s of MB)
<code>.cache/huggingface/</code> , <code>sentence-transformers/</code>	HuggingFace/ST model caches	Downloaded on first run
<code>venv/</code> , <code>.venv/</code>	Virtual environments	Machine-specific

9. Package Dependencies — Full Analysis

9.1 Runtime Dependencies

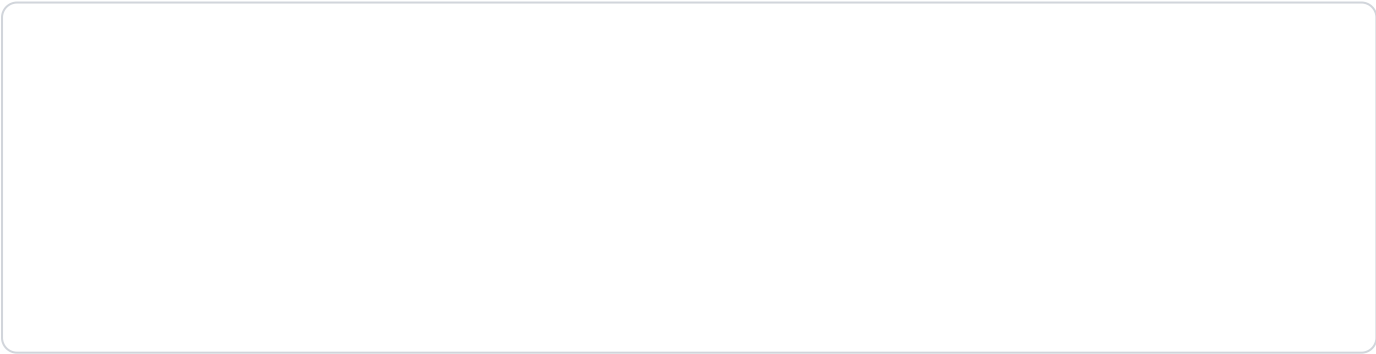
Package	Version	Category	Role in This Project
<code>fastapi</code>	Latest	Web Framework	Provides the async REST API framework. Handles routing, request validation (via Pydantic), and OpenAPI docs auto-generation. Powers all 4 endpoints.
<code>uvicorn[standard]</code>	Latest	ASGI Server	Production-grade async server that runs the FastAPI app. The <code>[standard]</code> extra includes <code>uvloop</code> for performance and <code>websockets</code> support.
<code>langchain</code>	Latest	AI Orchestration	Core framework providing abstractions for LLM chains, document models, prompt templates, and output parsers. Used throughout the entire AI pipeline.
<code>langchain-huggingface</code>	Latest	LLM Integration	Provides <code>HuggingFaceEndpoint</code> (connects to HF Inference API) and <code>ChatHuggingFace</code> (chat interface wrapper). Used in <code>RiskScorer</code> and <code>LegalNodes</code> .
<code>langchain-chroma</code>	Latest	Vector Store	Provides the <code>Chroma</code> LangChain wrapper around ChromaDB. Used by <code>VectorStoreManager</code> to embed, store, and search documents.
<code>langchain-text-splitters</code>	Latest	Text Processing	Provides the <code>TextSplitter</code> base class that <code>LegalClauseSplitter</code> extends with custom regex-based legal document splitting.
<code>langgraph</code>	Latest	Workflow Engine	Provides <code>StateGraph</code> and <code>END</code> for building the 3-node directed graph workflow. Enables stateful, composable AI pipelines with typed state.
<code>sentence-transformers</code>	Latest	Embeddings	Underlying library for <code>HuggingFaceEmbeddings</code> . Loads and runs the <code>all-MiniLM-L6-v2</code> model to generate 384-dimensional clause embeddings.
<code>huggingface-hub</code>	Latest	API Client	Python client for the HuggingFace Hub API. Handles authentication, model downloads, and inference API calls.
<code>chromadb</code>	Latest	Vector Database	Persistent vector database storing clause embeddings. Provides <code>PersistentClient</code> for disk-backed storage and cosine similarity search.
<code>pydantic</code>	Latest	Data Validation	Defines structured schemas (<code>RiskClause</code> , <code>RiskReport</code> , <code>QueryRequest</code>) with automatic validation. <code>PydanticOutputParser</code> uses these to parse LLM JSON output.
<code>python-dotenv</code>	Latest	Configuration	Loads environment variables from <code>.env</code> files. Used in <code>project_config.py</code> to read <code>HUGGINGFACEHUB_API_TOKEN</code> .

Package	Version	Category	Role in This Project
<code>tiktoken</code>	Latest	Tokenization	OpenAI's fast BPE tokenizer. Used internally by LangChain for token counting when splitting or truncating text to fit model context windows.
<code>python-multipart</code>	Latest	File Upload	Required by FastAPI for parsing <code>multipart/form-data</code> file uploads. Without it, <code>UploadFile</code> endpoints would fail.
<code>pypdf</code>	Latest	PDF Parsing	Pure-Python PDF reader. <code>DocumentLoader._parse_pdf()</code> uses <code>pypdf.PdfReader</code> to extract text from each page of uploaded PDFs.
<code>python-docx</code>	Latest	DOCX Parsing	Reads Microsoft Word <code>.docx</code> files. <code>DocumentLoader._parse_docx()</code> uses <code>docx.Document</code> to extract paragraph text.

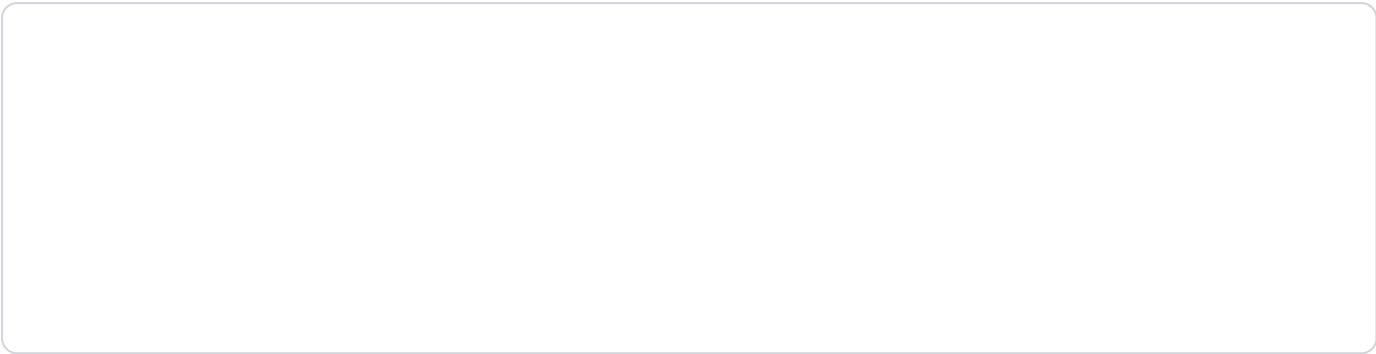
9.2 Development / Testing Dependencies

Package	Role in This Project
<code>pytest</code>	Test framework. Discovers and runs all 34 tests. Provides assertions, fixtures, parametrize, and rich error reporting.
<code>pytest-asyncio</code>	Enables <code>async def</code> test functions and <code>async</code> fixtures. Required because all FastAPI endpoint tests use <code>await</code> . Configured with <code>asyncio_mode = "auto"</code> in <code>pyproject.toml</code> .
<code>httpx</code>	Async HTTP client. Provides <code>ASGITransport</code> to send requests directly through the ASGI app without a running server — the standard way to test async FastAPI apps.

9.3 Dependency Relationship Map



10. Deployment Architecture



11. Security Considerations

Concern	Mitigation
API key exposure	Keys stored in <code>.env</code> (git-ignored) locally; stored in GitHub Secrets for CI/CD
Uploaded file safety	Files are parsed in-memory (<code>BytesIO</code>); no shell execution; format whitelist (TXT/PDF/DOCX only)
Dependency supply chain	<code>requirements.txt</code> pins to package names (consider adding version pins for production)
ChromaDB data	Local persistent storage; excluded from git. No PII leaves the server unless sent to HF Inference API.

12. Summary

The AI Legal Policy and Document Analyzer is a production-ready application that combines:

1. **Multi-format document ingestion** (PDF, DOCX, TXT) with regex-based legal clause splitting.
2. **Semantic search** via ChromaDB + Sentence-Transformer embeddings for relevant clause retrieval.
3. **Hybrid risk scoring** that merges LLM intelligence with deterministic keyword rules for reliability.
4. **LangGraph state-machine orchestration** for a clean, composable 3-node analysis pipeline.
5. **A comprehensive CI pipeline** with 34 fully-mocked pytest tests running in < 1 second.
6. **Automated deployment** to Hugging Face Spaces via Docker on every push to `main`.

The architecture follows separation of concerns across 5 distinct modules (`ingestion` , `retrieval` , `risk_engine` , `workflows` , `utils`), each independently testable and clearly bounded.